```
  private final StreamBridge streamBridge;

  public OrderService(BookClient bookClient,
    StreamBridge streamBridge, OrderRepository orderRepository
  ) {
    this.bookClient = bookClient;
    this.orderRepository = orderRepository;
    this.streamBridge = streamBridge;
  }

  ...

  private void publishOrderAcceptedEvent(Order order) {
    if (!order.status().equals(OrderStatus.ACCEPTED)) {
      return;                                                    ❶
    }
    var orderAcceptedMessage =
      new OrderAcceptedMessage(order.id());                      ❷
    log.info("Sending order accepted event with id: {}", order.id());
    var result = streamBridge.send("acceptOrder-out-0",
      orderAcceptedMessage);                                     ❸
    log.info("Result of sending data for order with id {}: {}",
      order.id(), result);
  }
}
```

❶ If the order is not accepted, it does nothing.

❷ Builds a message to notify that an order has been accepted

❸ Explicitly sends a message to the acceptOrder-out-0 binding

Since the data source is a REST endpoint, there is no `Supplier` bean we can register with Spring Cloud Function, and therefore there is no trigger for the framework to create the necessary bindings with RabbitMQ. Yet, in listing 10.23, `StreamBridge` is used to send data to an `acceptOrder-out-0` binding. Where does it come from? There is no `acceptOrder` function!

At startup time, Spring Cloud Stream will notice that `StreamBridge` wants to publish messages via an `acceptOrder-out-0` binding, and it will create one automatically. Similar to the bindings created from